HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO IOT VÀ ỨNG DỤNG

Giảng viên hướng dẫn	**: Nguyễn Quốc Uy**
Lớp	**: D23CNPM06**
Họ và tên	**: Lê Xuân Cường**
Mã sinh viên	**: B23DCCN111**

Hà Nội – 2026

1. TỔNG QUAN HỆ THỐNG..... 3
1.1. Tên đề tài..... 3
1.2. Bài toán đặt ra..... 3
1.3. Mục tiêu..... 4
1.4. Kiến trúc tổng quan hệ thống..... 4
2. USE CASE..... 5
2.1. Use Case tổng quan..... 5
2.2. UC01 - Đăng nhập hệ thống..... 6
2.2.1. Giao diện đăng nhập..... 6
2.2.2. Sequence Diagram đăng nhập..... 7
2.3. UC02 - Giám sát môi trường..... 7
2.3.1. Giao diện Dashboard..... 8
2.3.2. Sequence Diagram giám sát môi trường..... 9
2.4. UC03 - Điều khiển thiết bị..... 9
2.4.1. Giao diện điều khiển thiết bị..... 10
2.4.2. Sequence Diagram điều khiển thiết bị..... 11
2.5. UC04 - Tra cứu lịch sử cảm biến..... 11
2.5.1. Giao diện lịch sử cảm biến..... 12
2.5.2. Sequence Diagram tra cứu cảm biến..... 13
2.6. UC05 - Tra cứu lịch sử thao tác thiết bị..... 13
2.6.1. Giao diện lịch sử thiết bị..... 14
2.6.2. Sequence Diagram lịch sử thiết bị..... 14
2.7. UC06 - Tùy chỉnh hệ thống Smart Room..... 15
2.7.1. Giao diện Smart Custom..... 16
2.7.2. Sequence Diagram Smart Custom..... 17
2.8. Màn hình hỗ trợ và điều hướng..... 17
2.8.1. Profile..... 17
2.8.2. Điều hướng chung..... 18
3. DATABASE..... 18
3.1. Mô hình dữ liệu..... 18
Hình 2. ERD tổng thể cho Core IoT và Smart Custom..... 18
3.2. Bảng Users..... 19
3.3. Bảng Sensors..... 19
3.4. Bảng Devices..... 20
3.5. Bảng SensorData..... 21
3.6. Bảng ActionHistory..... 21

3.7. Bảng UserPreferences..... 23
3.8. Bảng Thresholds.....24
3.9. Bảng AutomationRules..... 25
3.10. Bảng AutomationConditions.....26
3.11. Bảng Scenes..... 27
3.12. Bảng SceneActions..... 28
4. API DOCUMENTATION..... 29
4.1. Nguyên tắc thiết kế API..... 29
4.2. Ánh xạ Use Case - Module - API đại diện.....30
4.3. Auth Module - UC01 Đăng nhập hệ thống..... 31
4.4. Sensor Module - UC02 và UC04..... 33
4.5. Device & Action Module - UC03 và UC05.....34
4.6. Smart Custom Module - UC06.....36
4.7. Realtime Communication.....39
4.8. Quy ước Response, Error và Pagination..... 40
4.9. Quy ước MQTT.....42
4.10. Quan hệ giữa các module..... 42
5. KẾT LUẬN.....43

1. TỔNG QUAN HỆ THỐNG

1.1. Tên đề tài

Xây dựng hệ thống IoT giám sát môi trường và điều khiển thiết bị trong phòng.

1.2. Bài toán đặt ra

Trong một không gian phòng, người dùng cần theo dõi các thông số môi trường quan trọng gồm nhiệt độ, độ ẩm và cường độ ánh sáng. Bên cạnh việc quan sát dữ liệu tại thời điểm hiện tại, hệ thống cần lưu lại lịch sử đo để phục vụ tra cứu, đánh giá xu hướng và kiểm tra lại các thời điểm bất thường. Đồng thời, người dùng cần có khả năng điều khiển thiết bị từ giao diện web và biết được trạng thái thật của phần cứng sau khi gửi lệnh.

Điểm quan trọng của hệ thống là phân biệt giữa hành động người dùng yêu cầu và trạng thái thực tế của thiết bị. Khi lệnh bật hoặc tắt vừa được gửi đi, trạng thái được đặt là “loading”. Chỉ sau khi phần cứng phản hồi, trạng thái mới được cập nhật thành “on” hoặc “off”. Cách xử lý này giúp giao diện phản ánh đúng tình trạng kết nối và hạn chế trường hợp người dùng tưởng rằng thiết bị đã hoạt động trong khi phần cứng không nhận được lệnh.

1.3. Mục tiêu

· Xây dựng giao diện web trực quan để giám sát nhiệt độ, độ ẩm và ánh sáng theo thời gian thực.

· Hiển thị biểu đồ lịch sử để người dùng quan sát xu hướng dữ liệu trong 24 giờ.

· Cho phép điều khiển hai thiết bị từ Dashboard và cập nhật trạng thái theo phản hồi phần cứng.

· Lưu, lọc và tìm kiếm lịch sử dữ liệu cảm biến.

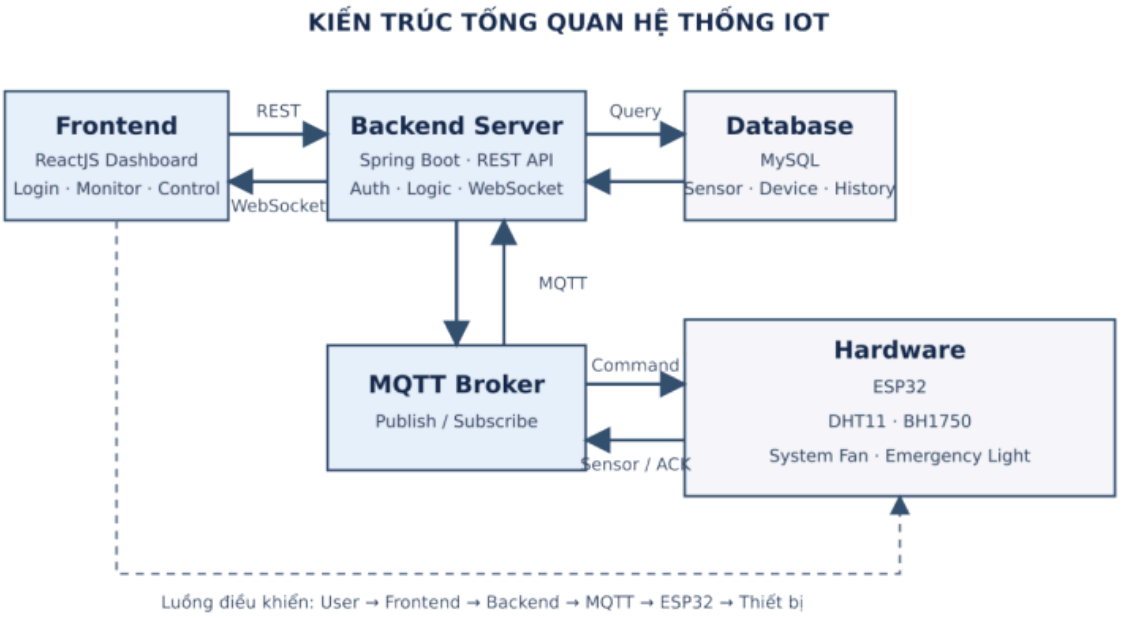
· Lưu lịch sử thao tác trên thiết bị gồm người thực hiện, thiết bị, action, status và thời gian.

· Mô tả rõ luồng giao tiếp giữa Frontend, Backend, Database, MQTT và Hardware.

· Tích hợp lớp tùy biến Smart Custom trên Figma/prototype nhằm tăng giá trị trải nghiệm, cho phép người dùng cá nhân hóa Dashboard, đặt ngưỡng cảnh báo, tạo Automation và sử dụng Scene nhanh.

1.4. Kiến trúc tổng quan hệ thống

Hệ thống được tổ chức theo mô hình gồm năm thành phần chính: Frontend, Backend Server, Database, MQTT Broker và Hardware. Kiến trúc này tách rõ lớp giao diện, xử lý nghiệp vụ, lưu trữ dữ liệu và giao tiếp thiết bị.



Hình 1.4.1 Kiến trúc tổng quan hệ thống IoT

Frontend được xây dựng bằng ReactJS, cung cấp giao diện đăng nhập, giám sát dữ liệu cảm biến, điều khiển thiết bị, tra cứu lịch sử và cấu hình Smart Custom. Frontend gửi yêu cầu tới Backend bằng REST API và nhận sự kiện thời gian thực qua WebSocket.

Backend Server sử dụng Spring Boot, chịu trách nhiệm xác thực, xử lý nghiệp vụ, truy vấn Database và làm cổng giao tiếp duy nhất với MQTT Broker. Database MySQL lưu người dùng, cảm biến, thiết bị, dữ liệu đo, lịch sử thao tác và các cấu hình tự động hóa.

Tầng Hardware sử dụng ESP32 kết nối DHT11, BH1750, System Fan và Emergency Light. ESP32 publish dữ liệu cảm biến và trạng thái xác nhận lên MQTT; đồng thời subscribe các lệnh điều khiển từ Backend.

Luồng giám sát: DHT11/BH1750 → ESP32 → MQTT Broker → Backend → Database/WebSocket → Frontend.

Luồng điều khiển: Người dùng → Frontend → Backend → MQTT Broker → ESP32 → thiết bị; trạng thái chỉ được cập nhật on/off sau khi Backend nhận ACK từ phần cứng.

2. USE CASE

2.1. Use Case tổng quan

Actor chính của hệ thống là Người dùng đã được cấp tài khoản. Sáu Use Case chính được lựa chọn để bao phủ các chức năng quan trọng trong thiết kế.

Mã	Use Case	Màn hình
UC01	Đăng nhập hệ thống	Login
UC02	Giám sát môi trường	Dashboard
UC03	Điều khiển thiết bị	Dashboard
UC04	Tra cứu lịch sử cảm biến	Data Sensor
UC05	Tra cứu lịch sử thao tác thiết bị	Action History
UC06	Tùy chỉnh hệ thống Smart Room (Smart Custom)	Smart Custom

2.2. UC01 - Đăng nhập hệ thống

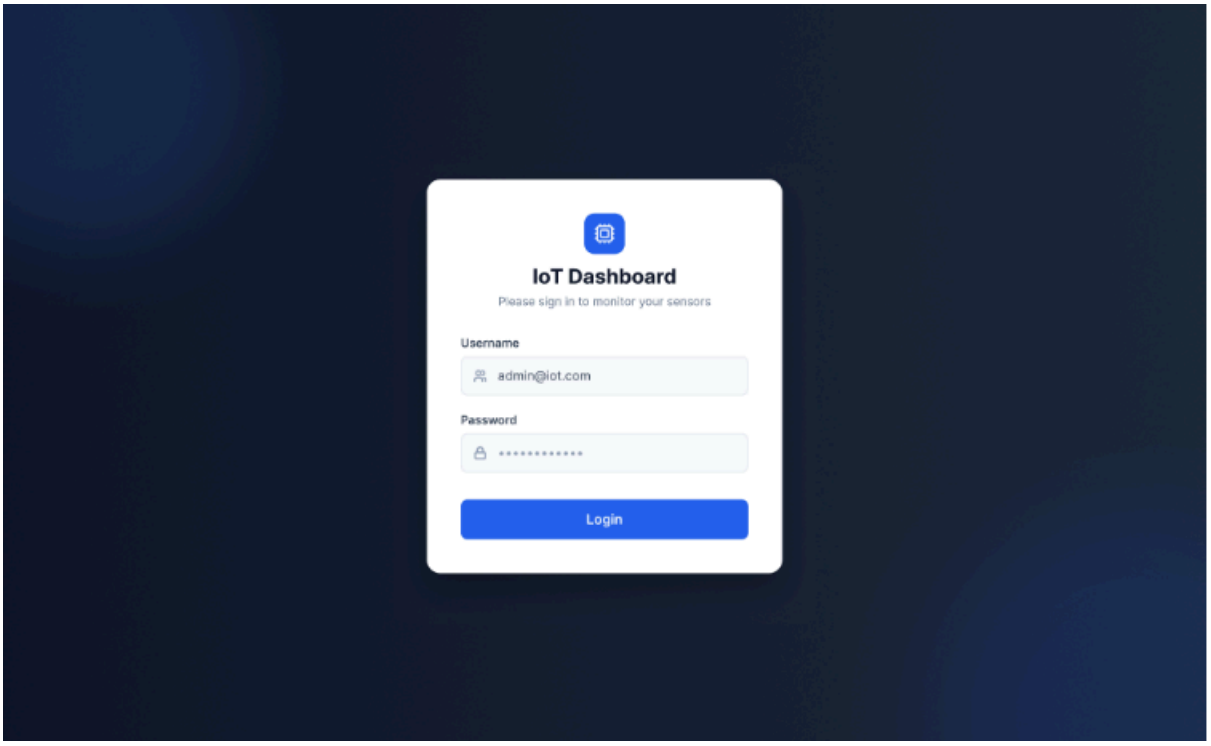
Thuộc tính	Mô tả
Mục tiêu	Xác thực người dùng trước khi truy cập các chức năng giám sát và điều khiển.
Actor	Người dùng.
Tiền điều kiện	Người dùng có tài khoản hợp lệ; Backend và Database hoạt động.
Hậu điều kiện	Tạo phiên đăng nhập/token và chuyển người dùng tới Dashboard.

Basic Flow (Luồng chính)	1) Mở trang Login. 2) Nhập tài khoản và mật khẩu. 3) Nhấn Login. 4) Frontend gửi yêu cầu xác thực. 5) Backend kiểm tra thông tin. 6) Nếu hợp lệ, hệ thống trả token/phiên đăng nhập và mở Dashboard.
Alternative Flow (Luồng thay thế/ngoại lệ)	Sai tài khoản hoặc mật khẩu: hiển thị thông báo lỗi và giữ người dùng ở trang Login.

2.2.1. Giao diện đăng nhập

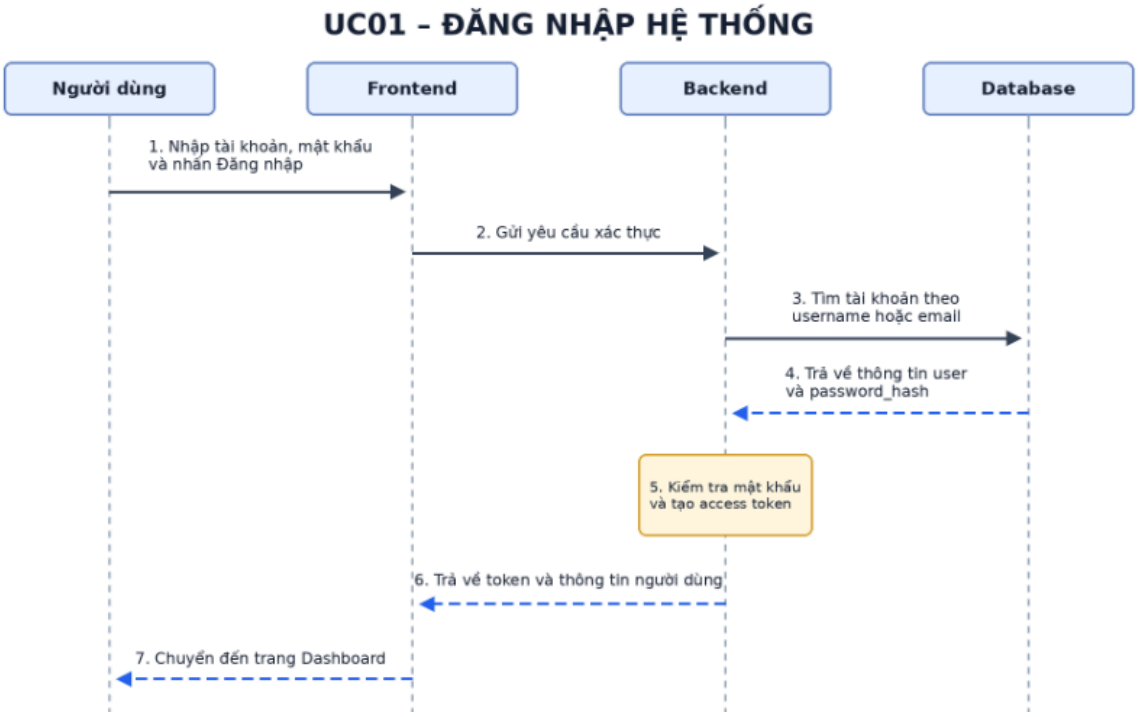
Màn hình Login sử dụng một thẻ đăng nhập đặt ở giữa giao diện. Form gồm trường tài khoản, trường mật khẩu và nút Login. Thiết kế hiện tại minh họa tài khoản dạng "admin@iot.com". Sau khi xác thực thành công, người dùng được chuyển tới Dashboard.

Thành phần	**Mô tả**
Username / Email	Nhập tài khoản người dùng.
Password	Nhập mật khẩu ở dạng che ký tự.
Login	Gửi thông tin xác thực tới Backend.



Hình 2.2.1. Giao diện đăng nhập

2.2.2. Sequence Diagram đăng nhập



Hình 2.2.2 Sequence Diagram UC01 - Đăng nhập hệ thống

Giải thích chi tiết

Sequence Diagram UC01 mô tả quá trình người dùng gửi thông tin đăng nhập từ Frontend tới Backend, Backend truy vấn tài khoản trong Database, kiểm tra mật khẩu và trả token/phiên đăng nhập. Khi xác thực thành công, Frontend chuyển người dùng tới Dashboard.

2.3. UC02 - Giám sát môi trường

Thuộc tính	**Mô tả**
Mục tiêu	Theo dõi đồng thời trạng thái môi trường hiện tại và xu hướng biến đổi của nhiệt độ, độ ẩm và ánh sáng trên Dashboard.
Actor	Người dùng đã đăng nhập.
Tiền điều kiện	Dashboard được mở; Backend nhận được dữ liệu cảm biến realtime và cơ sở dữ liệu có SensorData để dựng biểu đồ lịch sử.
Hậu điều kiện	Ba thẻ Temperature, Humidity, Light hiển thị giá trị mới nhất; biểu đồ Historical Trends hiển thị dữ liệu theo thời gian.

Basic Flow (Luồng chính)	1) Người dùng mở Dashboard. 2) Frontend lấy giá trị cảm biến mới nhất từ Backend. 3) Backend đọc dữ liệu realtime/giá trị gần nhất nhận từ MQTT. 4) Frontend cập nhật các thẻ Temperature, Humidity, Light, xu hướng và thời gian cập nhật. 5) Frontend gọi API lấy dữ liệu lịch sử 24 giờ. 6) Backend truy vấn SensorData và trả danh sách điểm dữ liệu theo timestamp. 7) Frontend dựng biểu đồ Historical Trends cho ba loại cảm biến.
Alternative Flow (Luồng thay thế/ngoại lệ)	Mất kết nối realtime: giữ giá trị cuối cùng và hiển thị trạng thái dữ liệu không còn realtime. Không có dữ liệu lịch sử: biểu đồ hiển thị trạng thái rỗng/chưa có dữ liệu.

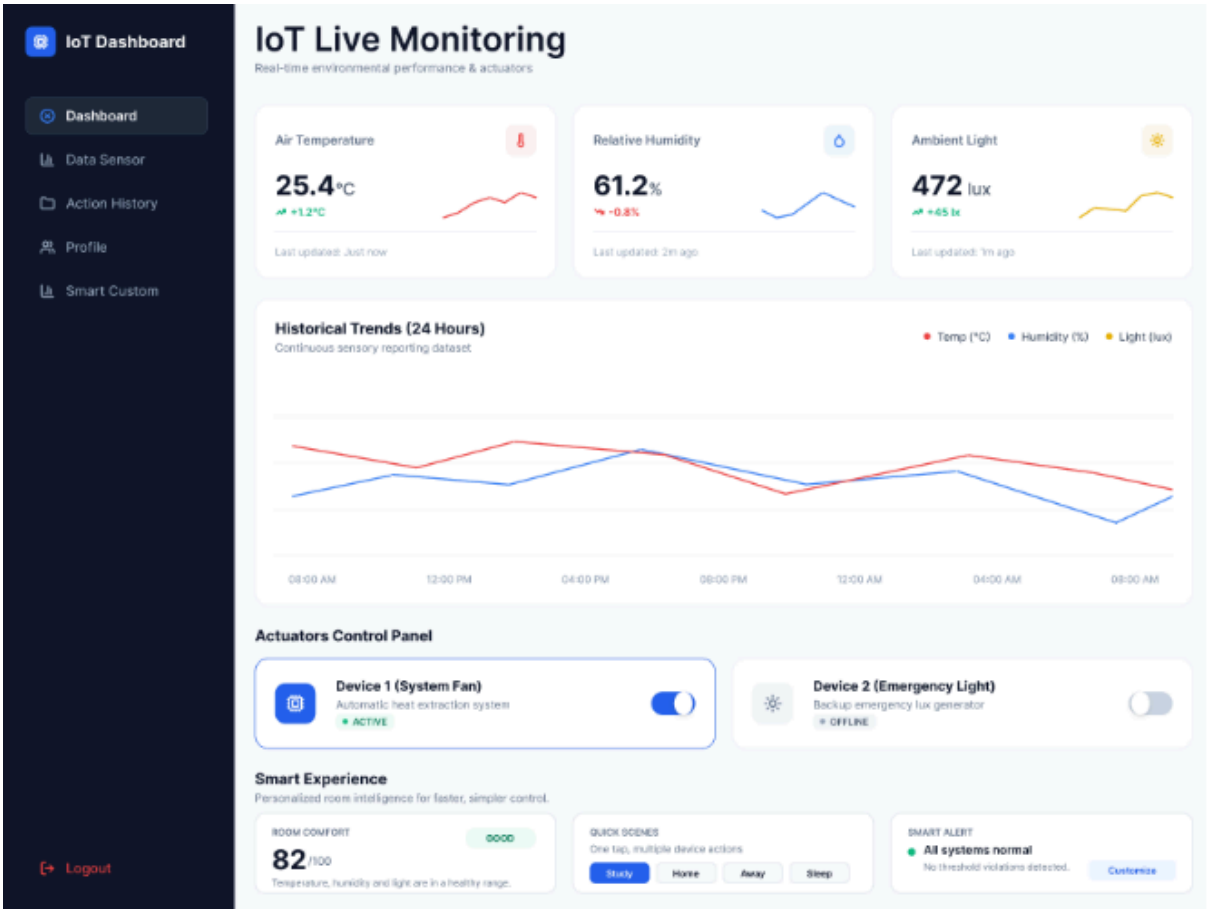
2.3.1. Giao diện Dashboard

Dashboard có tiêu đề “IoT Live Monitoring”, đóng vai trò trung tâm giám sát và điều khiển. Phần trên hiển thị ba thẻ thông số Air Temperature, Relative Humidity và Ambient Light, kèm giá trị, xu hướng và thời điểm cập nhật gần nhất.

Bên dưới là biểu đồ “Historical Trends (24 Hours)” và “Actuators Control Panel” cho System Fan và Emergency Light. Phiên bản Figma cập nhật còn bổ sung khu vực Smart Experience ở cuối Dashboard, gồm Room Comfort Score, Quick Scenes (Study/Home/Away/Sleep), Smart Alert và nút Customize để chuyển trực tiếp sang Smart Custom.

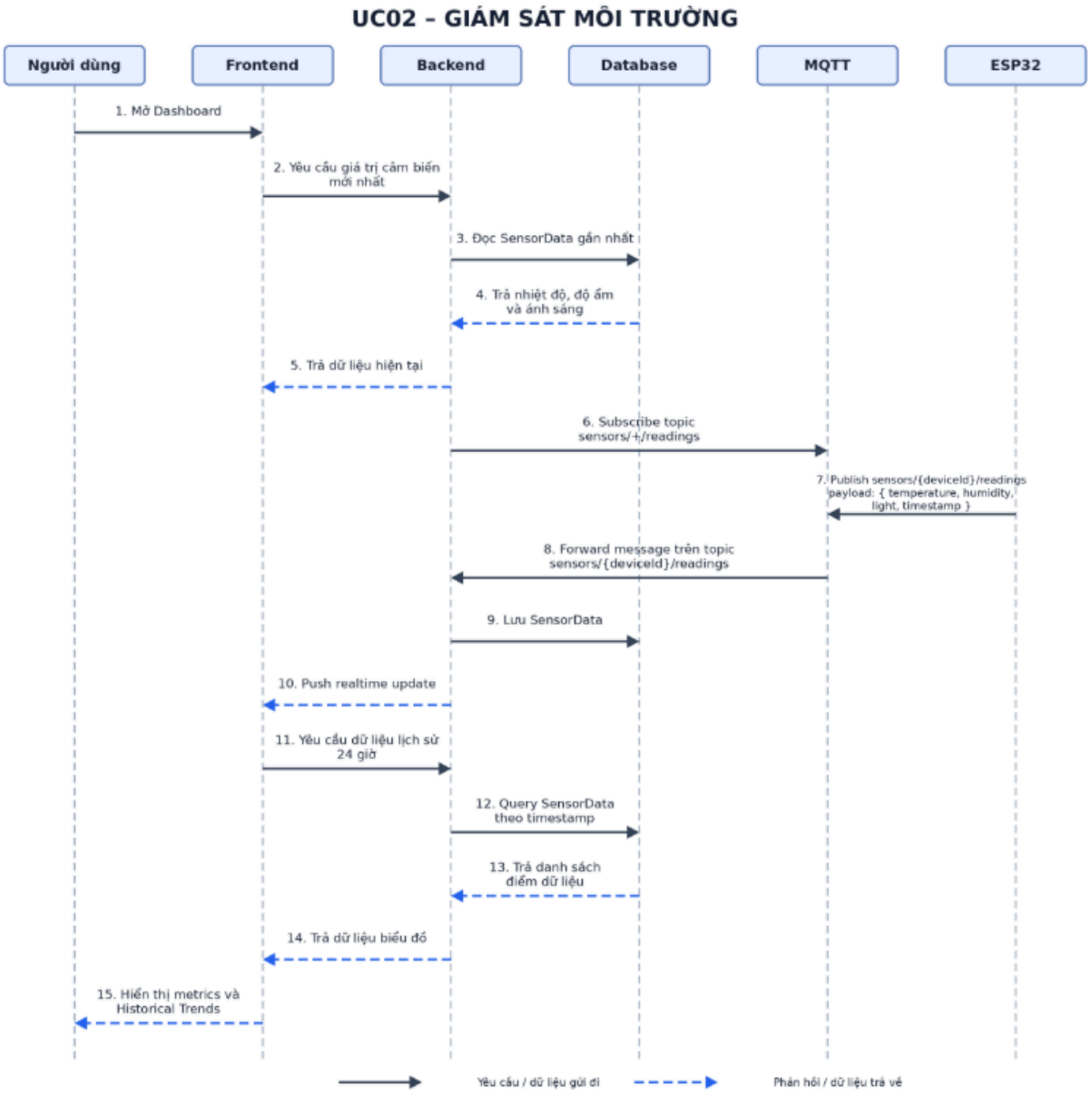
Khu vực	**Chức năng**
Realtime metrics	Hiển thị giá trị mới nhất của 3 cảm biến.

Historical Trends	Biểu diễn xu hướng dữ liệu trong 24 giờ.
System Fan	Thiết bị 1, có toggle và trạng thái ACTIVE/OFF/LOADING tùy phản hồi.
Emergency Light	Thiết bị 2, có toggle và trạng thái hoạt động.
Smart Experience	Comfort Score, Quick Scenes, Smart Alert và nút Customize để mở Smart Custom.



Hình 2.3.1 Giao diện dashboard

2.3.2. Sequence Diagram giám sát môi trường



Hình 2.3.2 Sequence Diagram UC02 - Giám sát môi trường

Giải thích chi tiết

Sequence Diagram UC02 thể hiện hai luồng kết hợp trên Dashboard: lấy giá trị cảm biến hiện tại và nhận dữ liệu realtime từ ESP32 qua MQTT, đồng thời truy vấn SensorData để dựng biểu đồ Historical Trends trong 24 giờ.

2.4. UC03 - Điều khiển thiết bị

Thuộc tính	Mô tả
Mục tiêu	Điều khiển System Fan hoặc Emergency Light từ web và phản ánh đúng trạng thái thật của phần cứng.
Actor	Người dùng đã đăng nhập.
Tiền điều kiện	Thiết bị tồn tại; Backend kết nối MQTT Broker; thiết bị đã được khai báo trong hệ thống.
Hậu điều kiện	Action History có bản ghi thao tác; status được cập nhật theo phản hồi thực tế từ thiết bị.
Basic Flow (Luồng chính)	1) Người dùng nhấn toggle ON/OFF trên Dashboard. 2) Frontend gửi action tới Backend. 3) Backend tạo bản ghi ActionHistory với status=loading. 4) Backend publish lệnh qua MQTT. 5) ESP32 nhận lệnh và điều khiển thiết bị. 6) ESP32 gửi phản hồi trạng thái. 7) Backend cập nhật status=on/off. 8) Frontend nhận trạng thái mới và cập nhật toggle/badge.
Alternative Flow (Luồng thay thế/ngoại lệ)	Không nhận phản hồi phần cứng: status không chuyển sang on/off; hệ thống giữ loading trong khoảng chờ và có thể chuyển timeout/error khi triển khai.

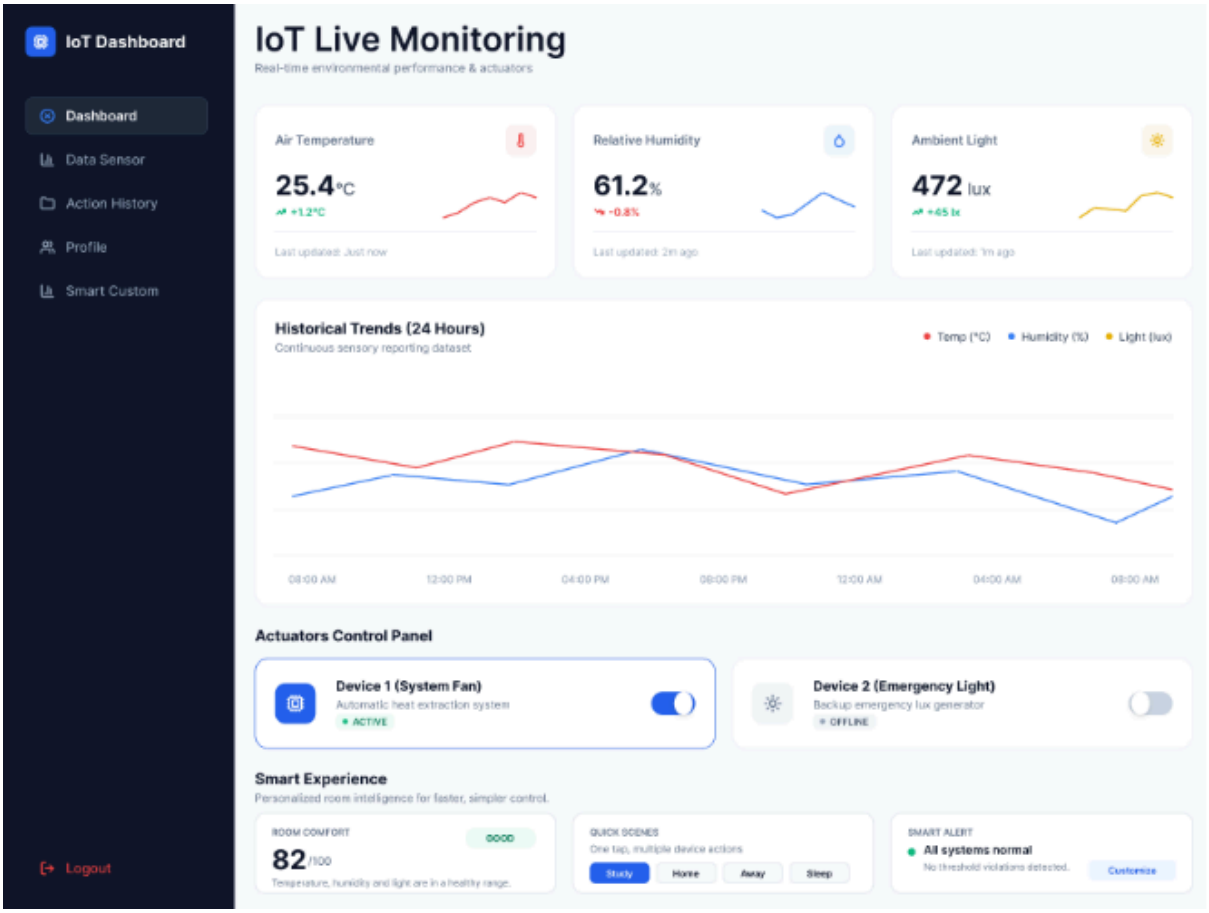
2.4.1. Giao diện điều khiển thiết bị

Dashboard có tiêu đề “IoT Live Monitoring”, đóng vai trò trung tâm giám sát và điều khiển. Phần trên hiển thị ba thẻ thông số Air Temperature, Relative Humidity và Ambient Light, kèm giá trị, xu hướng và thời điểm cập nhật gần nhất.

Bên dưới là biểu đồ “Historical Trends (24 Hours)” và “Actuators Control Panel” cho System Fan và Emergency Light. Phiên bản Figma cập nhật còn bổ sung khu vực Smart Experience ở cuối Dashboard, gồm Room Comfort Score,

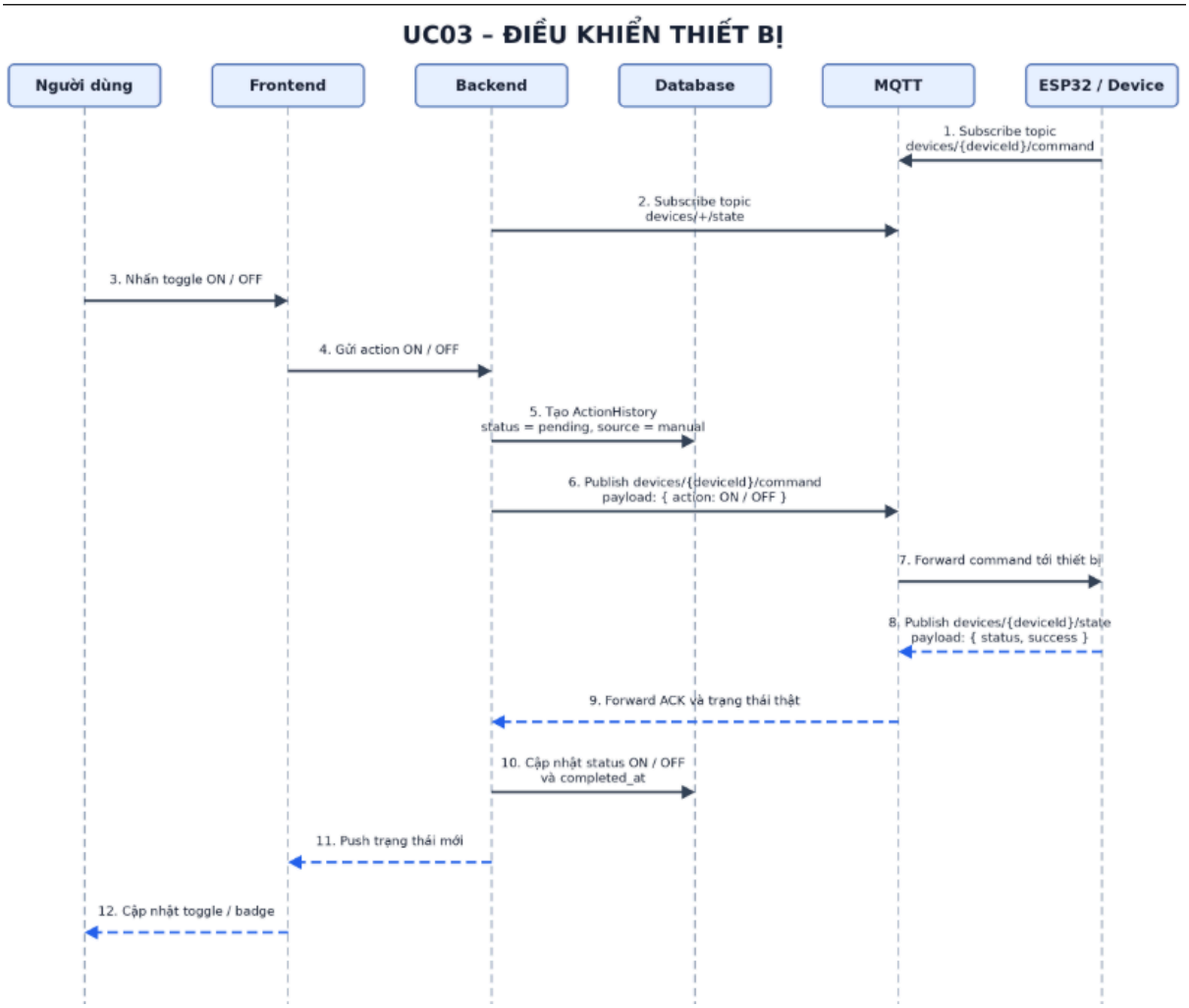
Quick Scenes (Study/Home/Away/Sleep), Smart Alert và nút Customize để chuyển trực tiếp sang Smart Custom.

Khu vực	**Chức năng**
Realtime metrics	Hiển thị giá trị mới nhất của 3 cảm biến.
Historical Trends	Biểu diễn xu hướng dữ liệu trong 24 giờ.
System Fan	Thiết bị 1, có toggle và trạng thái ACTIVE/OFF/LOADING tùy phản hồi.
Emergency Light	Thiết bị 2, có toggle và trạng thái hoạt động.
Smart Experience	Comfort Score, Quick Scenes, Smart Alert và nút Customize để mở Smart Custom.



Hình 2.4.1. Giao diện điều khiển

2.4.2. Sequence Diagram điều khiển thiết bị



Hình 2.4.2. Sequence Diagram UC03 - Điều khiển thiết bị

Giải thích chi tiết

Khi người dùng bật/tắt System Fan hoặc Emergency Light, Backend ghi ActionHistory với status=loading trước khi publish lệnh qua MQTT. Chỉ sau khi ESP32/Device phản hồi ACK, Backend mới cập nhật status thành on/off và đẩy trạng thái mới về giao diện.

2.5. UC04 - Tra cứu lịch sử cảm biến

Thuộc tính	Mô tả

Mục tiêu	Tìm lại dữ liệu cảm biến theo loại, thời gian hoặc điều kiện giá trị để phục vụ tra cứu và kiểm tra các thời điểm bất thường.
Actor	Người dùng đã đăng nhập.
Tiền điều kiện	Có dữ liệu trong SensorData.
Hậu điều kiện	Bảng Data Sensor hiển thị tập kết quả đúng điều kiện và hỗ trợ phân trang.
Basic Flow (Luồng chính)	1) Người dùng mở Data Sensor. 2) Chọn Sensor Type hoặc để All Sensors. 3) Nhập/chọn thời gian cần tra cứu. 4) Có thể đặt giới hạn giá trị. 5) Nhấn Search. 6) Backend tạo truy vấn theo điều kiện. 7) Frontend hiển thị ID, loại cảm biến, giá trị, đơn vị và timestamp.
Alternative Flow (Luồng thay thế/ngoại lệ)	Không có kết quả phù hợp: hiển thị danh sách rỗng và thông báo không tìm thấy dữ liệu.

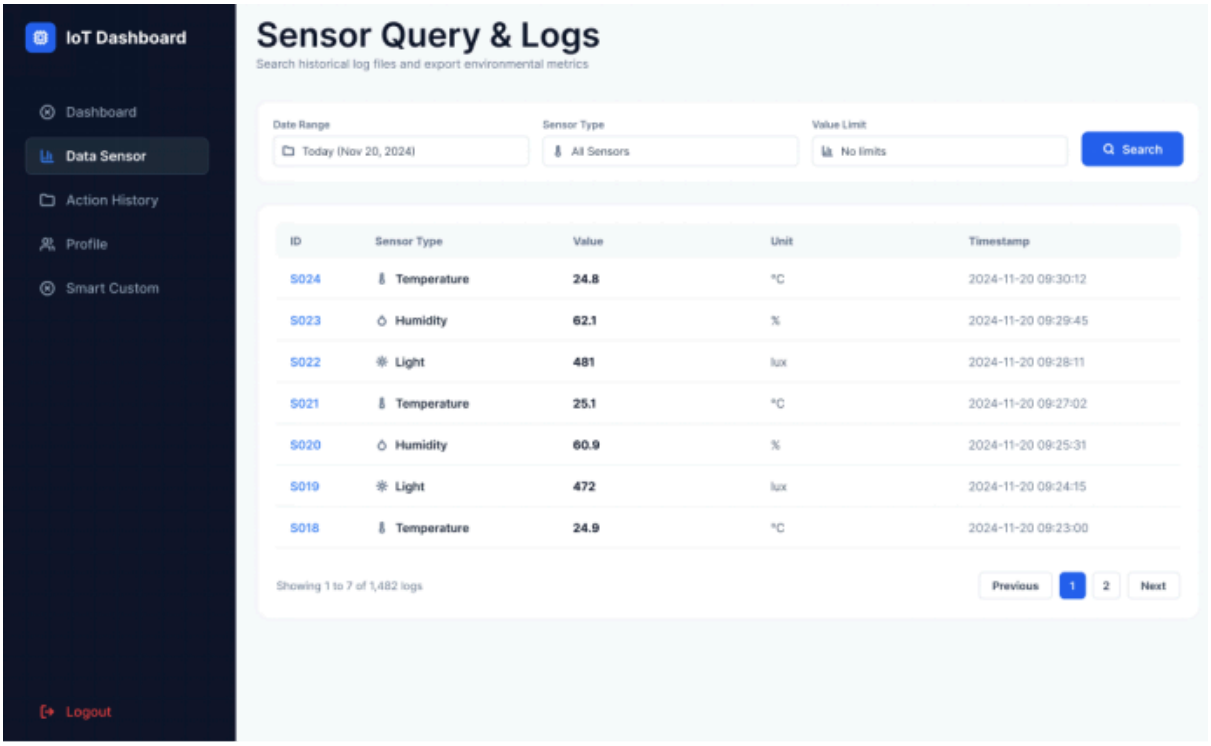
2.5.1. Giao diện lịch sử cảm biến

Data Sensor là màn hình tra cứu lịch sử cảm biến. Thanh bộ lọc trong thiết kế gồm Date Range, Sensor Type, Value Limit và nút Search. Bảng dữ liệu gồm các cột ID, Sensor Type, Value, Unit và Timestamp. Thiết kế có phân trang Previous, số trang và Next.

Về nghiệp vụ, hệ thống cần hỗ trợ tìm kiếm theo thời gian với định dạng năm-tháng-ngày giờ:phút:giây. Người dùng có thể nhập một phần chuỗi thời gian để mở rộng phạm vi tìm kiếm, ví dụ bỏ giây để lấy toàn bộ dữ liệu trong phút hoặc bỏ phút để lấy toàn bộ dữ liệu trong giờ.

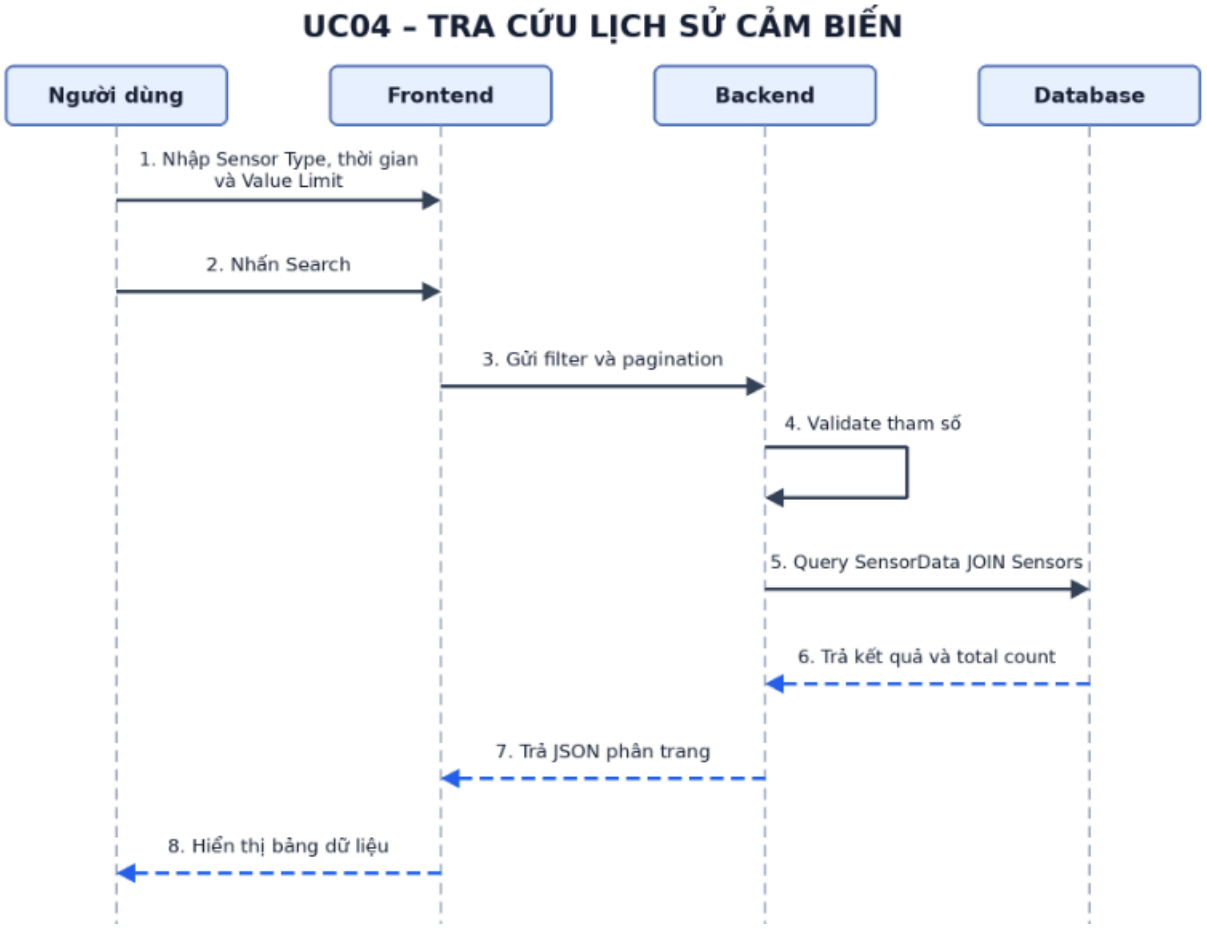
Trường lọc	**Ý nghĩa**

Date Range / Time	Lọc theo thời gian đo.
Sensor Type	All Sensors, Temperature, Humidity hoặc Light.
Value Limit	Giới hạn/lọc dữ liệu theo giá trị nếu cần.
Search	Thực thi truy vấn và cập nhật bảng.



Hình 2.5.1. Giao diện lịch sử cảm biến

2.5.2. Sequence Diagram tra cứu cảm biến



Hình 2.5.2. Sequence Diagram UC04 - Tra cứu lịch sử cảm biến

Giải thích chi tiết

Sequence Diagram UC04 mô tả luồng tìm kiếm trên Data Sensor. Người dùng chọn loại cảm biến, thời gian và giới hạn giá trị; Frontend gửi các điều kiện lọc tới Backend, Backend truy vấn SensorData kết hợp Sensors và trả kết quả phân trang cho giao diện.

2.6. UC05 - Tra cứu lịch sử thao tác thiết bị

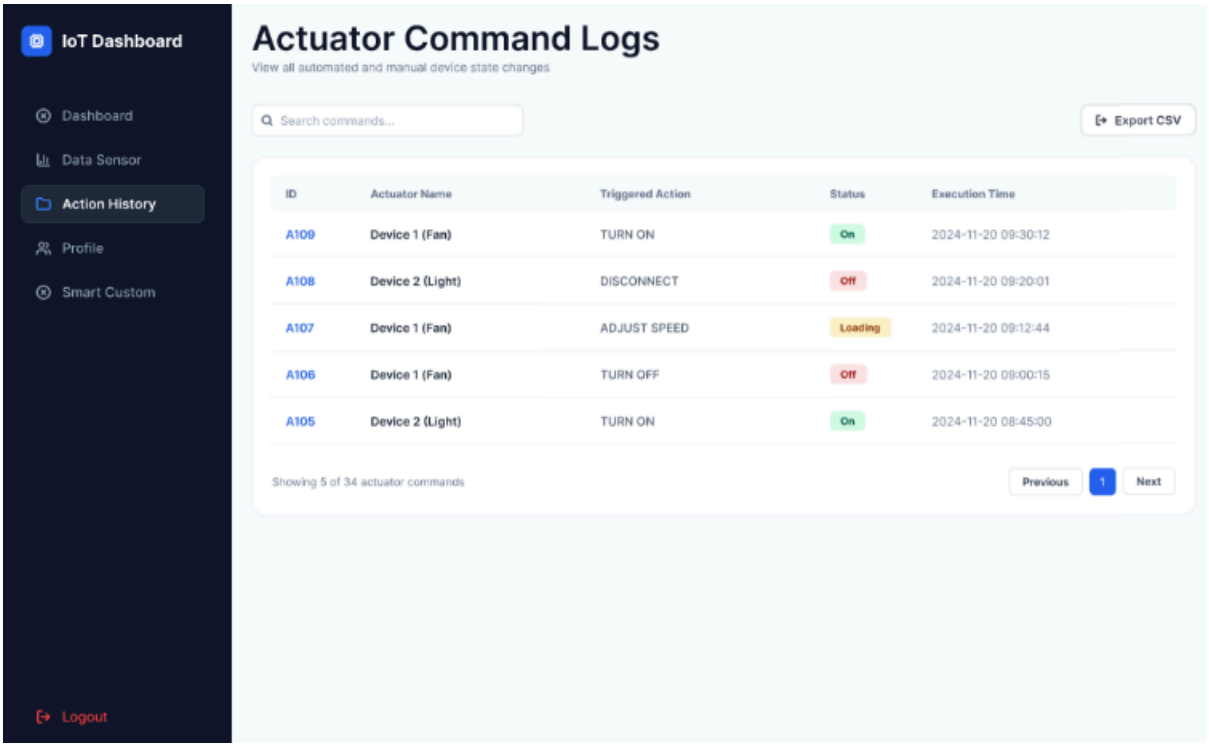
Thuộc tính	**Mô tả**

Mục tiêu	Kiểm tra thiết bị nào đã được điều khiển, action nào được gửi, trạng thái thực tế và thời gian thực hiện.
Actor	Người dùng đã đăng nhập.
Tiền điều kiện	ActionHistory có dữ liệu.
Hậu điều kiện	Bảng Action History hiển thị kết quả lọc/tìm kiếm; có thể phân trang hoặc xuất CSV khi chức năng được triển khai.
Basic Flow (Luồng chính)	1) Người dùng mở Action History. 2) Nhập từ khóa hoặc chọn điều kiện lọc theo thiết bị/action/status/time. 3) Frontend gọi API. 4) Backend truy vấn ActionHistory. 5) Frontend hiển thị ID, Actuator Name, Triggered Action, Status và Execution Time.
Alternative Flow (Luồng thay thế/ngoại lệ)	Không có dữ liệu phù hợp: bảng rỗng, không phát sinh lỗi hệ thống.

2.6.1. Giao diện lịch sử thiết bị

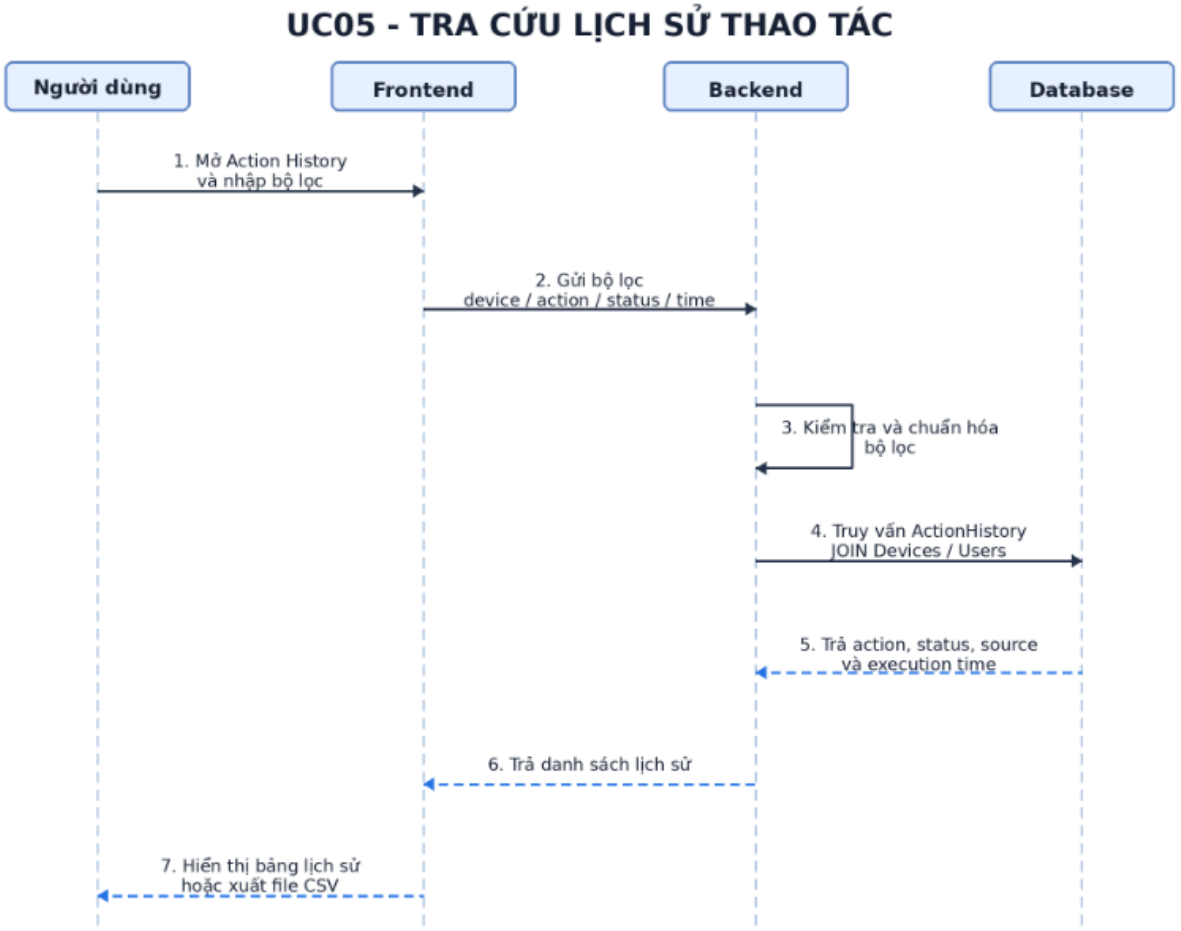
Action History quản lý lịch sử lệnh điều khiển thiết bị. Bản thiết kế có ô “Search commands...”, nút Export CSV và bảng với các cột ID, Actuator Name, Triggered Action, Status và Execution Time. Status được thể hiện dưới dạng badge, gồm On, Off và Loading.

Thiết kế minh họa nhiều loại action như TURN ON, TURN OFF, DISCONNECT và ADJUST SPEED. Đối với phạm vi bài tập cơ bản, action chính cần hỗ trợ là ON/OFF. Trạng thái Loading được dùng khi Backend đã nhận yêu cầu nhưng chưa có phản hồi xác nhận từ phần cứng.



Hình 2.6.1. Giao diện lịch sử thiết bị

2.6.2. Sequence Diagram lịch sử thiết bị



Hình 2.6.2. Sequence Diagram UC05 - Tra cứu lịch sử thao tác thiết bị

Giải thích chi tiết

Sequence Diagram UC05 mô tả quá trình tra cứu Action History theo thiết bị, action, status, thời gian hoặc từ khóa. Backend truy vấn ActionHistory kết hợp thông tin thiết bị/người dùng và trả danh sách lịch sử để Frontend hiển thị hoặc xuất CSV.

2.7. UC06 - Tùy chỉnh hệ thống Smart Room

Thuộc tính	Mô tả

Mục tiêu	Cho phép người dùng cá nhân hóa cách hệ thống hiển thị, cảnh báo và tự động vận hành theo nhu cầu riêng mà không cần thay đổi mã nguồn.
Actor	Người dùng đã đăng nhập.
Tiền điều kiện	Màn Smart Custom truy cập được; tài khoản, cảm biến và thiết bị đã được khai báo trong hệ thống.
Hậu điều kiện	Cấu hình cá nhân được lưu theo tài khoản; Dashboard, Threshold/Alert, Automation và Quick Scene sử dụng cấu hình mới.
Basic Flow (Luồng chính)	1) Người dùng mở Smart Custom từ sidebar hoặc nút Customize trên Dashboard. 2) Hệ thống hiển thị cấu hình hiện tại của tài khoản. 3) Người dùng bật/tắt widget và chọn Chart Range cho Dashboard. 4) Người dùng cấu hình Threshold & Alert cho Temperature, Humidity và Ambient Light. 5) Người dùng có thể tạo Automation Rule dạng IF điều kiện cảm biến → THEN hành động thiết bị và bật Active. 6) Người dùng chọn hoặc cấu hình Quick Scene như Study, Home, Away, Sleep. 7) Người dùng nhấn Save/Save Automation. 8) Backend lưu cấu hình theo tài khoản. 9) Khi quay lại Dashboard, hệ thống áp dụng cấu hình mới; lệnh phát sinh từ Automation/Scene vẫn được ghi ActionHistory và chờ ACK phần cứng.
Alternative Flow (Luồng thay thế/ngoại lệ)	Giá trị ngưỡng/rule không hợp lệ: không cho lưu và hiển thị thông báo. Lưu cấu hình thất bại: giữ cấu hình cũ. Nếu Automation/Scene điều khiển thiết bị nhưng phần cứng offline, trạng thái lệnh giữ loading rồi chuyển timeout/error theo cơ chế triển khai.

2.7.1. Giao diện Smart Custom

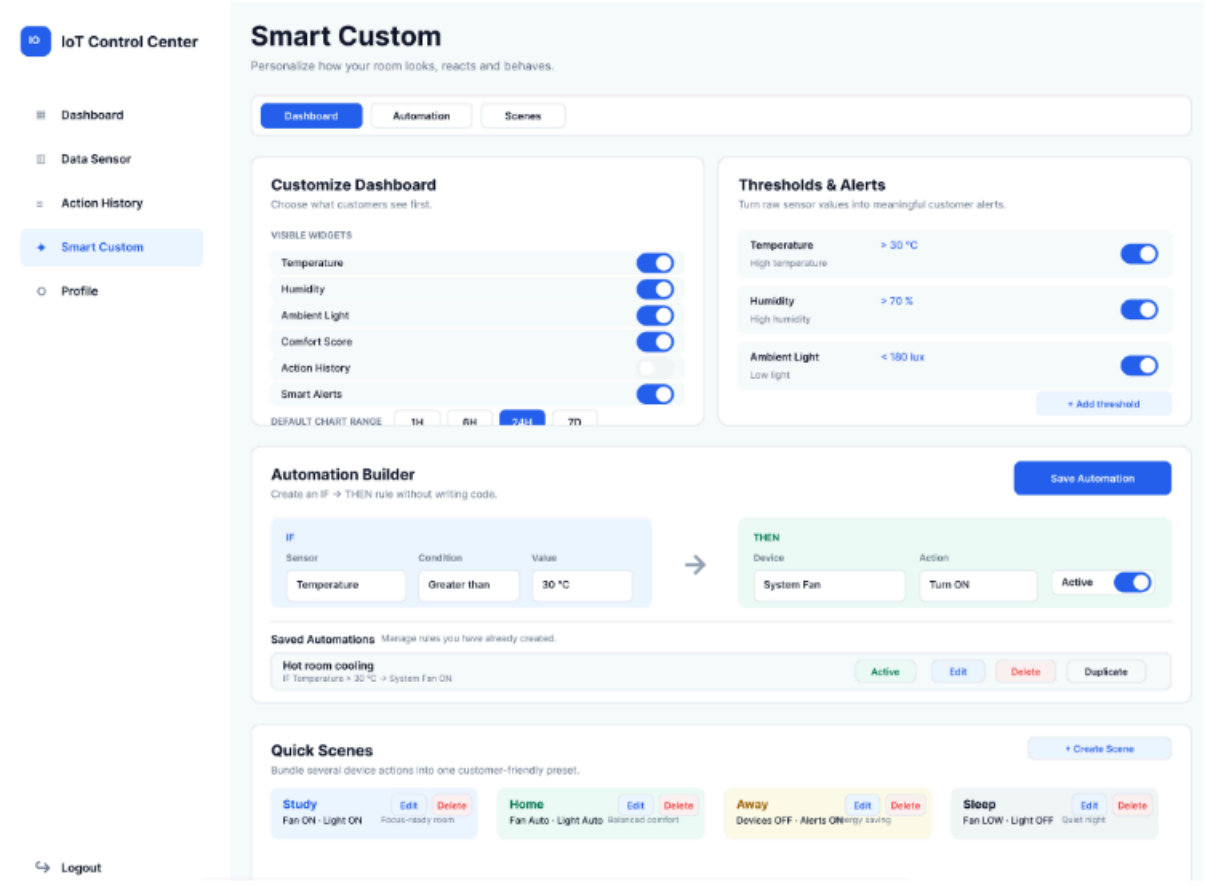
Smart Custom có ba nhóm thao tác chính Dashboard, Automation và Scenes. Sidebar của các màn hình chính và nút Customize trên Dashboard đều được nối tới màn hình này.

· Customize Dashboard: bật/tắt 6 widget chính và chọn khoảng biểu đồ 1H, 6H, 24H hoặc 7D.

· Thresholds & Alerts: đặt ngưỡng Temperature > 30°C, Humidity > 70% và Ambient Light < 180 lux; có thể bật/tắt từng ngưỡng.

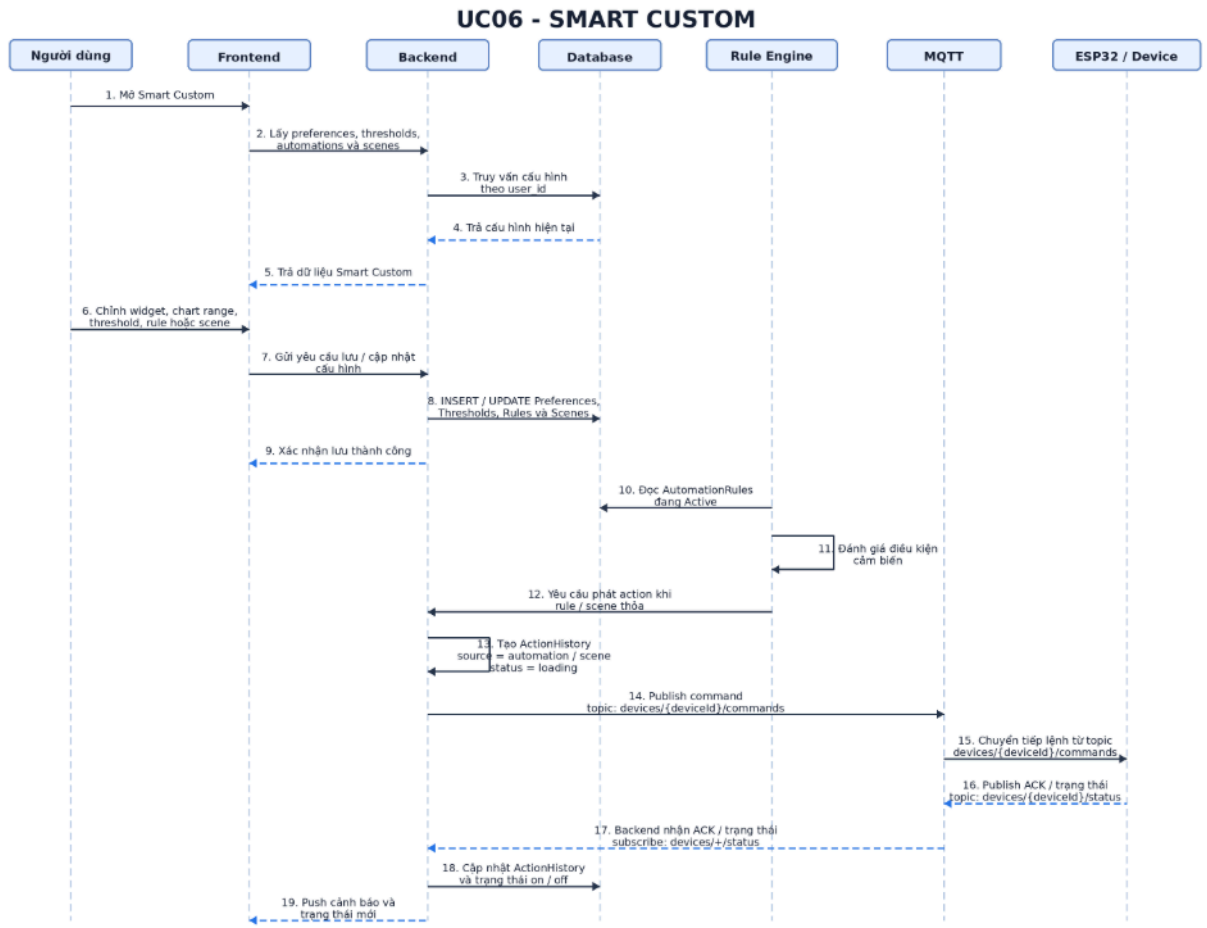
· Automation Builder: tạo rule no-code IF Temperature > 30°C → System Fan Turn ON, có Active và Save Automation.

· Quick Scenes: các preset Study, Home, Away và Sleep giúp thực hiện nhiều hành động chỉ bằng một thao tác.



Hình 2.7.1. Giao diện Smart Custom

2.7.2. Sequence Diagram Smart Custom



Hình 2.7.2. Sequence Diagram UC06 - Tùy chỉnh hệ thống Smart Room

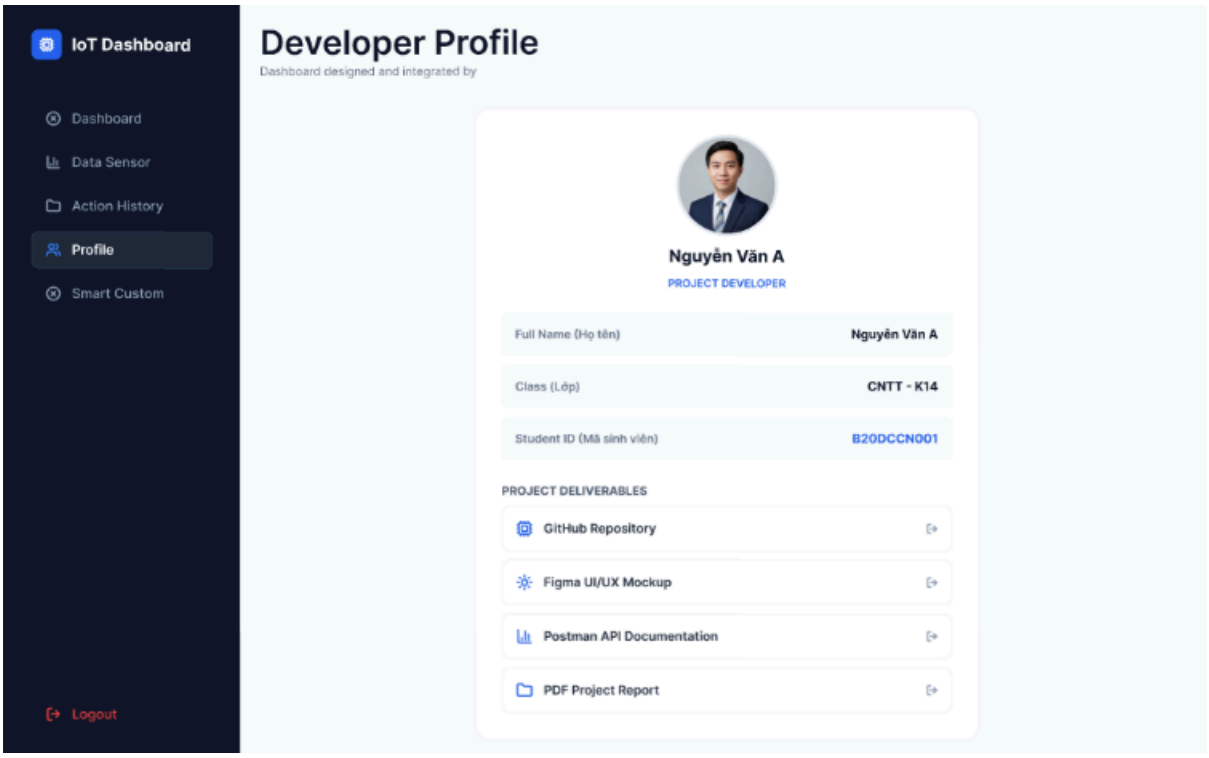
Giải thích chi tiết

Sequence Diagram UC06 gồm hai phần chính. Phần thứ nhất là đọc và lưu UserPreferences, Thresholds, AutomationRules/AutomationConditions, Scenes/SceneActions theo tài khoản. Phần thứ hai thể hiện cách Rule Engine hoặc Scene phát sinh lệnh điều khiển: Backend tạo ActionHistory với source=automation/scene và status=loading, gửi lệnh qua MQTT, chờ ACK từ ESP32/Device rồi mới cập nhật trạng thái thật và thông báo cho Frontend.

2.8. Màn hình hỗ trợ và điều hướng

2.8.1. Profile

Profile hiển thị ảnh sinh viên, họ tên, lớp, mã sinh viên và nhóm liên kết bàn giao dự án. Thiết kế có bốn mục Project Deliverables: GitHub Repository, Figma UI/UX Mockup, Postman API Documentation và PDF Project Report.



Hình 2.8.1. Giao diện profile

2.8.2. Điều hướng chung

Sau đăng nhập, người dùng có thể chuyển giữa Dashboard, Data Sensor, Action History, Smart Custom và Profile bằng sidebar. Smart Custom đã được nối đầy đủ trong prototype; từ màn hình Smart Custom người dùng cũng có thể quay lại các màn hình còn lại. Nút Logout đặt ở cuối sidebar dùng để kết thúc phiên đăng nhập và quay về màn hình Login.

3. DATABASE

3.1. Mô hình dữ liệu

Cơ sở dữ liệu được thiết kế thành hai nhóm nhưng sử dụng chung một mô hình quan hệ. Nhóm Core IoT gồm Users, Sensors, Devices, SensorData và ActionHistory để phục vụ đăng nhập, giám sát môi trường, điều khiển thiết bị và lưu lịch sử. Nhóm Smart Custom gồm UserPreferences, Thresholds, AutomationRules, AutomationConditions, Scenes và SceneActions để lưu cấu hình cá nhân, ngưỡng cảnh báo, luật tự động hóa và các Scene có thể tạo, sửa, xóa. Tổng cộng mô hình gồm 11 bảng.

Quan hệ chính:

• Users 1:1 UserPreferences.

• Users 1:N Thresholds, AutomationRules, Scenes và ActionHistory.

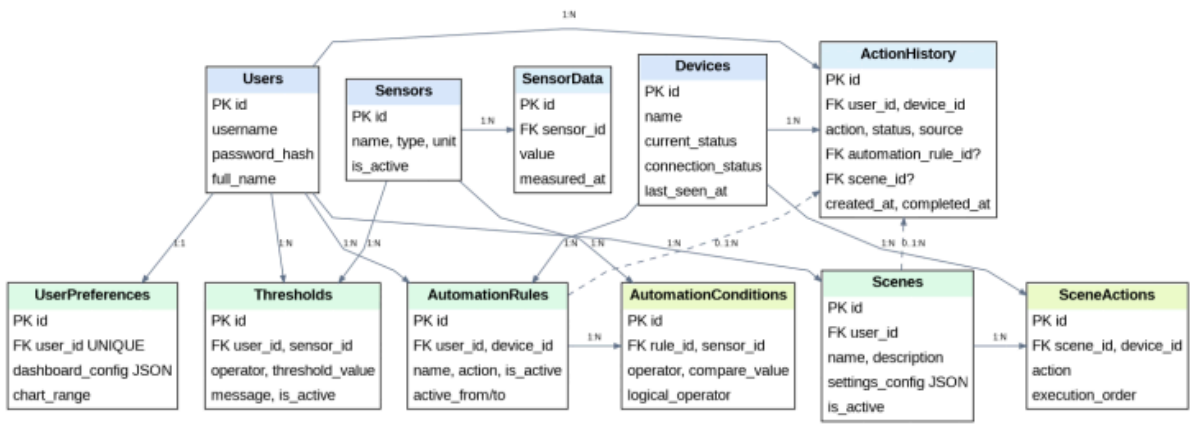
• Sensors 1:N SensorData, Thresholds và AutomationConditions.

• Devices 1:N AutomationRules, SceneActions và ActionHistory.

• AutomationRules 1:N AutomationConditions.

• Scenes 1:N SceneActions.

Mô hình dữ liệu tổng thể: Core IoT + Smart Custom



Hình 3.1. ERD tổng thể cho Core IoT và Smart Custom

3.2. Bảng Users

Trường	Kiểu gợi ý	Ràng buộc	Mô tả
id	BIGINT	PK, AUTO INCREMENT	Khóa chính người dùng.
username	VARCHAR(100)	UNIQUE, NOT NULL	Tên đăng nhập/email.
password_hash	VARCHAR(255)	NOT NULL	Mật khẩu đã băm.
full_name	VARCHAR(150)	NOT NULL	Họ tên người dùng.
created_at	DATETIME	NOT NULL	Thời điểm tạo tài khoản.

3.3. Bảng Sensors

Trường	Kiểu gợi ý	Ràng buộc	Mô tả
id	BIGINT	PK, AUTO INCREMENT	Mã cảm biến.
name	VARCHAR(100)	NOT NULL	Tên cảm biến.

type	VARCHAR(30)	NOT NULL	temperature/humidity/light.
unit	VARCHAR(20)	NOT NULL	°C, %, lux.
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Cho biết cảm biến đang được sử dụng.
created_at	DATETIME	NOT NULL	Thời điểm thêm cảm biến.

3.4. Bảng Devices

Trường	**Kiểu gợi ý**	**Ràng buộc**	**Mô tả**
id	BIGINT	PK, AUTO INCREMENT	Mã thiết bị.
name	VARCHAR(120)	NOT NULL	System Fan/Emergency Light hoặc tên tùy chỉnh.
current_status	VARCHAR(20)	NOT NULL	Trạng thái vận hành: on/off/loading.
connection_status	VARCHAR(20)	NOT NULL	Trạng thái kết nối: online/offline.

last_seen_at	DATETIME	NULL	Lần gần nhất phần cứng phản hồi.
created_at	DATETIME	NOT NULL	Thời điểm thêm thiết bị.

Việc tách current_status và connection_status giúp hệ thống phân biệt "thiết bị đang tắt" với "thiết bị đang mất kết nối". Đây là hai trạng thái khác nhau về nghiệp vụ và đều được thể hiện trên giao diện.

3.5. Bảng SensorData

Trường	**Kiểu gợi ý**	**Ràng buộc**	**Mô tả**
id	BIGINT	PK, AUTO INCREMENT	Mã bản ghi.
sensor_id	BIGINT	FK → Sensors.id, INDEX	Cảm biến tạo dữ liệu.
value	DECIMAL(10,2)	NOT NULL	Giá trị đo.
measured_at	DATETIME	NOT NULL, INDEX	Thời điểm đo.

3.6. Bảng ActionHistory

Trường	Kiểu gợi ý	Ràng buộc	Mô tả
id	BIGINT	PK, AUTO INCREMENT	Mã lịch sử thao tác.
user_id	BIGINT	FK → Users.id	Người sở hữu/thực hiện thao tác.
device_id	BIGINT	FK → Devices.id	Thiết bị được điều khiển.
action	VARCHAR(20)	NOT NULL	ON/OFF hoặc action mở rộng.
status	VARCHAR(20)	NOT NULL	loading/on/off/timeout/error.
source	VARCHAR(20)	NOT NULL	manual/automation/scene.
automation_rule_id	BIGINT	FK → AutomationRules.id, NULL	Rule phát sinh lệnh nếu source=automation.
scene_id	BIGINT	FK → Scenes.id, NULL	Scene phát sinh lệnh nếu source=scene.
created_at	DATETIME	NOT NULL, INDEX	Thời điểm gửi lệnh.

completed_at	DATETIME	NULL	Thời điểm nhận phản hồi cuối cùng từ phần cứng.

Quy tắc nghiệp vụ: action là ý định điều khiển, còn status là trạng thái đã được xác nhận. Vì vậy action=ON có thể đi kèm status=loading trong thời gian chờ ACK. Hai trường source và source_id tương ứng giúp Action History phân biệt lệnh do người dùng bấm trực tiếp, Automation hay Scene sinh ra.

3.7. Bảng UserPreferences

Trường	**Kiểu gợi ý**	**Ràng buộc**	**Mô tả**
id	BIGINT	PK, AUTO INCREMENT	Mã cấu hình.
user_id	BIGINT	FK → Users.id, UNIQUE	Mỗi tài khoản có một cấu hình Dashboard chính.
dashboard_config	JSON	NOT NULL	Danh sách widget bật/tắt và cấu hình hiển thị.
chart_range	VARCHAR(10)	NOT NULL	1H/6H/24H/7D.
created_at	DATETIME	NOT NULL	Thời điểm tạo cấu hình.

updated_at	DATETIME	NOT NULL	Thời điểm cập nhật gần nhất.

Ví dụ dashboard_config có thể lưu trạng thái hiển thị của Temperature, Humidity, Ambient Light, Comfort Score, Action History và Smart Alerts. JSON phù hợp ở đây vì cấu trúc widget có thể thay đổi mà không cần thêm cột mới mỗi lần mở rộng giao diện.

3.8. Bảng Thresholds

Trường	**Kiểu gợi ý**	**Ràng buộc**	**Mô tả**
id	BIGINT	PK, AUTO INCREMENT	Mã ngưỡng cảnh báo.
user_id	BIGINT	FK → Users.id	Người tạo ngưỡng.
sensor_id	BIGINT	FK → Sensors.id	Cảm biến áp dụng.
operator	VARCHAR(10)	NOT NULL	>, <, >=, <=, =.
threshold_value	DECIMAL(10,2)	NOT NULL	Giá trị ngưỡng.
message	VARCHAR(255)	NULL	Nội dung cảnh báo, ví dụ High temperature.

is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Bật/tắt ngưỡng.
created_at	DATETIME	NOT NULL	Thời điểm tạo.
updated_at	DATETIME	NOT NULL	Thời điểm sửa gần nhất.

Bảng này phù hợp với khu vực Thresholds & Alerts trên Figma, ví dụ Temperature > 30°C, Humidity > 70% và Ambient Light < 180 lux.

3.9. Bảng AutomationRules

Trường	**Kiểu gợi ý**	**Ràng buộc**	**Mô tả**
id	BIGINT	PK, AUTO INCREMENT	Mã Automation Rule.
user_id	BIGINT	FK → Users.id	Người tạo rule.
name	VARCHAR(150)	NOT NULL	Tên rule.
device_id	BIGINT	FK → Devices.id	Thiết bị được điều khiển khi rule thỏa mãn.
action	VARCHAR(20)	NOT NULL	Hành động, ví dụ ON/OFF.

is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Active/Inactive.
active_from	TIME	NULL	Giờ bắt đầu hiệu lực nếu có.
active_to	TIME	NULL	Giờ kết thúc hiệu lực nếu có.
created_at	DATETIME	NOT NULL	Thời điểm tạo.
updated_at	DATETIME	NOT NULL	Thời điểm chỉnh sửa gần nhất.

AutomationRules chỉ lưu thông tin chung và hành động đầu ra. Điều kiện được tách sang AutomationConditions để hỗ trợ Edit/Delete rule và mở rộng nhiều điều kiện AND/OR mà không phải sửa cấu trúc bảng.

3.10. Bảng AutomationConditions

Trường	**Kiểu gợi ý**	**Ràng buộc**	**Mô tả**
id	BIGINT	PK, AUTO INCREMENT	Mã điều kiện.
rule_id	BIGINT	FK → AutomationRules.id	Rule chứa điều kiện.

sensor_id	BIGINT	FK → Sensors.id	Cảm biến được kiểm tra.
operator	VARCHAR(10)	NOT NULL	>, <, >=, <=, =.
compare_value	DECIMAL(10,2)	NOT NULL	Giá trị so sánh.
logical_operator	VARCHAR(10)	NULL	AND/OR; điều kiện đầu tiên có thể để NULL.
sort_order	INT	NOT NULL, DEFAULT 1	Thứ tự đánh giá/hiển thị điều kiện.

Ví dụ một rule có thể gồm Temperature > 30°C AND Humidity > 70%, sau đó System Fan = ON.

3.11. Bảng Scenes

Trường	**Kiểu gợi ý**	**Ràng buộc**	**Mô tả**
id	BIGINT	PK, AUTO INCREMENT	Mã Scene.
user_id	BIGINT	FK → Users.id	Người sở hữu Scene.

name	VARCHAR(100)	NOT NULL	Study/Home/Away/Sleep hoặc tên tùy chỉnh.
description	VARCHAR(255)	NULL	Mô tả ngắn cho Scene.
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Cho phép sử dụng Scene.
created_at	DATETIME	NOT NULL	Thời điểm tạo.
updated_at	DATETIME	NOT NULL	Thời điểm chỉnh sửa gần nhất.

3.12. Bảng SceneActions

Trường	**Kiểu gợi ý**	**Ràng buộc**	**Mô tả**
id	BIGINT	PK, AUTO INCREMENT	Mã hành động trong Scene.
scene_id	BIGINT	FK → Scenes.id	Scene chứa hành động.
device_id	BIGINT	FK → Devices.id	Thiết bị cần điều khiển.

action	VARCHAR(20)	NOT NULL	ON/OFF hoặc action hỗ trợ khác.
execution_order	INT	NOT NULL, DEFAULT 1	Thứ tự thực thi khi Scene có nhiều hành động.

Tách SceneActions khỏi Scenes giúp một Scene chứa nhiều thiết bị và hỗ trợ đúng chức năng Create/Edit/Delete trên giao diện. Ví dụ Study có thể chứa Fan ON và Light ON dưới hai bản ghi SceneActions khác nhau.

4. API DOCUMENTATION

4.1. Nguyên tắc thiết kế API

API được tổ chức theo domain nghiệp vụ thay vì tách cứng một API cho từng Use Case. Một module có thể phục vụ nhiều Use Case có cùng dữ liệu và trách nhiệm, trong khi mỗi Use Case vẫn có một API đại diện để dễ đối chiếu với Chương 3 và Sequence Diagram ở Chương 5.

Các quy ước chung:

• Base URL: /api/v1.

• Dữ liệu trao đổi giữa Frontend và Backend sử dụng JSON.

• Các API nghiệp vụ yêu cầu Bearer Token; riêng API đăng nhập không yêu cầu token.

• REST API xử lý các thao tác request/response; WebSocket/Socket.IO được dùng để đẩy dữ liệu realtime từ Backend tới Frontend.

• MQTT chỉ dùng trong lớp giao tiếp Backend ↔ ESP32/Hardware, không để Frontend publish trực tiếp tới thiết bị.

• Lệnh điều khiển thiết bị được xử lý bất đồng bộ: Backend trả 202 Accepted và status=loading, sau đó mới cập nhật on/off khi nhận ACK từ phần cứng.

Kiến trúc module API:

Frontend

↕ REST / WebSocket

Backend API /api/v1

├─ Auth Module → Users

├─ Sensor Module → Sensors, SensorData

├─ Device & Action Module → Devices, ActionHistory ↔ MQTT ↔ ESP32/Device

└─ Smart Custom Module → UserPreferences, Thresholds, AutomationRules, AutomationConditions, Scenes, SceneActions

↓

Rule Engine

4.2. Ánh xạ Use Case - Module - API đại diện

Mã	Use Case	Module	API đại diện
UC01	Đăng nhập hệ thống	Auth Module	POST /api/v1/auth/login

UC02	Giám sát môi trường	Sensor Module	GET /api/v1/sensors/latest
UC03	Điều khiển thiết bị	Device & Action Module	POST /api/v1/devices/{id}/actions
UC04	Tra cứu lịch sử cảm biến	Sensor Module	GET /api/v1/sensors/data
UC05	Tra cứu lịch sử thao tác	Device & Action Module	GET /api/v1/actions
UC06	Smart Custom	Smart Custom Module	GET /api/v1/smart-custom

Bảng 6.1. Ánh xạ Use Case với module và API đại diện

Việc ánh xạ trên giúp tài liệu vẫn nhìn rõ theo Use Case, nhưng Backend không bị chia vụn thành sáu module độc lập. UC02 và UC04 cùng thuộc Sensor Module; UC03 và UC05 cùng thuộc Device & Action Module; UC06 sử dụng Smart Custom Module và khi cần điều khiển thiết bị sẽ gọi qua Device & Action Module.

4.3. Auth Module - UC01 Đăng nhập hệ thống

Trách nhiệm: xác thực tài khoản và cấp token cho phiên đăng nhập.

API đại diện:

POST /api/v1/auth/login

Request body:

```
{
  "username": "admin@iot.com",
  "password": "********"
}
```

Response thành công:

```
{
  "success": true,
  "data": {
    "token": "<access-token>",
    "user": {
      "id": 1,
      "fullName": "Le Xuan Cuong"
    }
  },
  "message": "Login successful"
}
```

Module truy vấn bảng Users, so sánh mật khẩu với password_hash và chỉ trả token khi xác thực thành công. Các API ở những module còn lại sử dụng token này để nhận diện người dùng.

4.4. Sensor Module - UC02 và UC04

Trách nhiệm: quản lý dữ liệu cảm biến hiện tại, dữ liệu lịch sử và cung cấp dữ liệu cho Dashboard/Data Sensor.

Các endpoint chính:

GET /api/v1/sensors/latest

Mục đích: API đại diện cho UC02, trả giá trị mới nhất của Temperature, Humidity và Ambient Light để dựng các thẻ realtime khi Dashboard được mở.

GET /api/v1/sensors/data

Mục đích: API đại diện cho UC04 và đồng thời có thể tái sử dụng để dựng Historical Trends trên Dashboard.

Query params đề xuất: type, time, valueMin, valueMax, page, limit.

Ví dụ:

GET /api/v1/sensors/data?type=temperature&time=2026-08-17%2010:30&page=1&limit=20

Response phân trang:


```
{
  "success": true,
  "data": [
    {
```

```
      "id": 1024,
      "sensorType": "temperature",
      "value": 28.4,
      "unit": "°C",
      "timestamp": "2026-08-17 10:30:12"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 1482,
    "totalPages": 75
  }
}
```


Đối với dữ liệu realtime, Backend nhận sensor reading từ MQTT, lưu SensorData rồi phát event sensor.updated qua WebSocket. Frontend không cần liên tục gọi GET để kiểm tra dữ liệu mới.

4.5. Device & Action Module - UC03 và UC05

Trách nhiệm: quản lý thiết bị, nhận mọi yêu cầu điều khiển, ghi ActionHistory và là cổng duy nhất được phép gửi command tới MQTT. Lệnh có thể đến từ người dùng, Automation hoặc Scene nhưng đều đi qua module này để bảo đảm cùng một cơ chế loading → ACK → on/off.

API đại diện UC03:

POST /api/v1/devices/{deviceId}/actions

Request body:

```
{
  "action": "ON"
}
```

Response ngay sau khi Backend tiếp nhận:

```
{
  "success": true,
  "data": {
    "actionId": "A110",
    "deviceId": 1,
    "action": "ON",
    "status": "loading",
    "source": "manual"
  },
  "message": "Command accepted and sent to MQTT"
}
```

HTTP Status: 202 Accepted. Backend tạo ActionHistory trước khi publish MQTT. Khi ESP32 phản hồi, Backend cập nhật status, completed_at và phát event device.status.changed tới Frontend.

API kiểm tra một command:

GET /api/v1/actions/{actionId}

API đại diện UC05:

GET /api/v1/actions

Query params đề xuất: deviceId, action, status, source, from, to, page, limit.

Endpoint này phục vụ màn Action History và có thể mở rộng thêm /api/v1/actions/export để xuất CSV.

4.6. Smart Custom Module - UC06

Trách nhiệm: quản lý toàn bộ cấu hình cá nhân của người dùng gồm Customize Dashboard, Threshold & Alert, Automation và Scene. Module này không publish MQTT trực tiếp. Khi Automation hoặc Scene tạo ra hành động thiết bị, Smart Custom Module chuyển yêu cầu sang Device & Action Module để ghi ActionHistory và xử lý ACK thống nhất.

API đại diện để tải toàn bộ màn Smart Custom:

GET /api/v1/smart-custom

Response có thể trả đồng thời preferences, thresholds, automations và scenes của tài khoản hiện tại, giúp Frontend chỉ cần một request khi mở màn hình.

Các nhóm endpoint chi tiết:

Preferences

GET /api/v1/preferences

PUT /api/v1/preferences

Thresholds

GET /api/v1/thresholds

POST /api/v1/thresholds

PUT /api/v1/thresholds/{id}

DELETE /api/v1/thresholds/{id}

Automations

GET /api/v1/automations

POST /api/v1/automations

PUT /api/v1/automations/{id}

DELETE /api/v1/automations/{id}

PATCH /api/v1/automations/{id}/status

Scenes

GET /api/v1/scenes

POST /api/v1/scenes

PUT /api/v1/scenes/{id}

DELETE /api/v1/scenes/{id}

POST /api/v1/scenes/{id}/execute

Ví dụ tạo Automation:

POST /api/v1/automations

```
{
  "name": "Hot and humid room",
  "conditions": [
    {"sensorId": 1, "operator": ">", "value": 30},
    {"logicalOperator": "AND", "sensorId": 2, "operator": ">", "value": 70}
  ],
  "deviceId": 1,
  "action": "ON",
  "isActive": true,
  "activeFrom": "08:00",
  "activeTo": "18:00"
}
```

Ví dụ kích hoạt Scene:

POST /api/v1/scenes/3/execute

Response:

```
{
```

```
  "success": true,
  "data": {
    "sceneId": 3,
    "name": "Study",
    "status": "executing",
    "actions": [
      {"actionId": "A201", "deviceId": 1, "status": "loading"},
      {"actionId": "A202", "deviceId": 2, "status": "loading"}
    ]
  }
}
```


Việc tách Scene và Automation thành tài nguyên CRUD giúp giao diện hiện tại hỗ trợ đúng các thao tác Create/Edit/Delete/Active mà không phải sửa cấu trúc API khi bổ sung thêm rule hoặc Scene mới.

4.7. Realtime Communication

REST API phù hợp với thao tác truy vấn và cập nhật cấu hình, nhưng dữ liệu IoT thay đổi liên tục nên Backend cần một kênh realtime riêng để đẩy sự kiện tới Frontend.

Các event đề xuất:

• sensor.updated: có dữ liệu cảm biến mới.

• device.status.changed: thiết bị nhận lệnh và thay đổi trạng thái thật.

• alert.triggered: một Threshold bị vượt ngưỡng.

• automation.executed: một Automation Rule đã được kích hoạt.

• scene.executed: một Scene đã được thực thi.

Luồng giao tiếp tổng quát:

ESP32 → MQTT → Backend → WebSocket → Frontend.

Frontend chỉ sử dụng REST để gửi yêu cầu hoặc truy vấn lịch sử, không polling liên tục để lấy trạng thái realtime.

4.8. Quy ước Response, Error và Pagination

Response thành công:

```
{
  "success": true,
  "data": {},
  "message": "Operation completed successfully"
}
```

Response lỗi:

```
{
  "success": false,
  "error": {
    "code": "DEVICE_OFFLINE",
    "message": "The device is currently offline",
```

```
    "field": null
  }
}
```


Các API danh sách sử dụng chung cấu trúc pagination gồm page, limit, total và totalPages. Cách chuẩn hóa này giúp Frontend xử lý SensorData, ActionHistory, Automations và Scenes bằng cùng một quy ước.

HTTP Status	Ý nghĩa
200 OK	Lấy/cập nhật dữ liệu thành công.
201 Created	Tạo resource mới thành công.
202 Accepted	Lệnh thiết bị đã được tiếp nhận, đang chờ ACK phần cứng.
400 Bad Request	Dữ liệu đầu vào hoặc tham số không hợp lệ.
401 Unauthorized	Chưa đăng nhập hoặc token không hợp lệ.
404 Not Found	Không tìm thấy resource.

409 Conflict	Xung đột trạng thái/dữ liệu, ví dụ resource trùng hoặc thao tác không hợp lệ.
500 Internal Server Error	Lỗi Backend/Database/MQTT ngoài dự kiến.

Bảng 6.2. Quy ước HTTP Status dùng trong hệ thống

4.9. Quy ước MQTT

MQTT nằm dưới lớp API và chỉ phục vụ giao tiếp Backend với phần cứng.

• iot/sensors/data: ESP32 → Backend, gửi nhiệt độ, độ ẩm và ánh sáng.

• iot/devices/{id}/command: Backend → ESP32, gửi lệnh điều khiển.

• iot/devices/{id}/status: ESP32 → Backend, phản hồi trạng thái thật/ACK.

Nguyên tắc quan trọng là Frontend không biết MQTT topic và không điều khiển ESP32 trực tiếp. Mọi thao tác phải đi qua Backend Module tương ứng để kiểm tra quyền, validate dữ liệu, ghi lịch sử và đồng bộ trạng thái.

4.10. Quan hệ giữa các module

Auth Module chịu trách nhiệm xác thực. Sensor Module sở hữu dữ liệu Sensors/SensorData. Device & Action Module là điểm duy nhất phát lệnh xuống hardware và quản lý ActionHistory. Smart Custom Module sở hữu cấu hình tùy biến, Rule và Scene nhưng ủy quyền việc điều khiển thiết bị cho Device & Action Module. Nhờ đó, luồng REST, Database, MQTT và

WebSocket được phân lớp rõ ràng, giảm phụ thuộc chéo giữa các chức năng và thuận lợi khi triển khai Backend theo module.

5. KẾT LUẬN

Bài thực hành 1 đã xác định được phạm vi và thiết kế nền tảng cho hệ thống IoT giám sát môi trường và điều khiển thiết bị trong phòng. Thiết kế Figma hiện gồm Login, Dashboard, Data Sensor, Action History, Smart Custom và Profile. Sáu Use Case chính được tổ chức gồm: UC01 Đăng nhập, UC02 Giám sát môi trường, UC03 Điều khiển thiết bị, UC04 Tra cứu lịch sử cảm biến, UC05 Tra cứu lịch sử thao tác thiết bị và UC06 Tùy chỉnh hệ thống Smart Room (Smart Custom).

Việc đưa Smart Custom thành UC06 giúp Figma, đặc tả nghiệp vụ và hướng triển khai Backend thống nhất với nhau. Trong UC06, người dùng có thể cá nhân hóa Dashboard, đặt Threshold & Alert, tạo Automation Rule và sử dụng Quick Scene. Các chức năng này tạo điểm khác biệt hướng tới khách hàng nhưng vẫn kế thừa cơ chế tin cậy của phần cốt lõi: mọi lệnh thiết bị đều được ghi ActionHistory, chuyển qua loading và chỉ xác nhận ON/OFF sau khi phần cứng phản hồi.

Ở giai đoạn tiếp theo, ưu tiên là kết nối giao diện với dữ liệu thật qua RESTful API/MQTT, triển khai Database và chương trình ESP32, sau đó hiện thực UserPreferences, Rule Engine và Scene execution cho Smart Custom. Khi các phần này được nối với nhau, hệ thống có thể demo trọn vẹn từ tùy chỉnh cá nhân → cảm biến vượt ngưỡng → tự động bật thiết bị → phản hồi trạng thái trên Dashboard.